

Đặc điểm và ứng dụng của thuật toán di truyền

Zhang Ning

Nguồn gốc: Characteristics and applications of genetic algorithms

I0I China candidate team papers / 2002 / Zhang Ning.pdf

Từ khóa. Thuật toán di truyền; di truyền; đột biến; nhiễm sắc thể; gen; quần thể.

Tóm Tắt

Thuật toán di truyền là một ngành mới được phát triển từ việc mô phỏng cơ chế tiến hóa của sự sống trên máy tính, dựa trên thuyết tiến hóa Darwin. Nó tiến hành tìm kiếm, tính toán và giải bài toán theo các quy luật tiến hóa tự nhiên như thích nghi thì tồn tại, ưu thắng liệt bại.

Phần thứ nhất của bài viết giới thiệu các khái niệm cơ bản của thuật toán di truyền. Phần thứ hai giới thiệu nguyên lý của thuật toán di truyền và ba phép toán: chọn lọc, lai ghép, đột biến. Phần thứ ba tập trung giới thiệu cách cài đặt cụ thể của ba phép toán này cùng các ví dụ đơn giản, chủ yếu thể hiện quá trình thực hiện thuật toán di truyền. Phần thứ tư giới thiệu hai bài toán cụ thể, đều thuộc lớp NP-đầy đủ, và cách dùng thuật toán di truyền để giải chúng cũng như một số vấn đề cơ bản khi cài đặt.

Khi giới thiệu nguyên lý và các phép toán của thuật toán di truyền, bài viết cũng phân tích một số vấn đề cơ bản xuất hiện trong ứng dụng. Điều này có ý nghĩa hướng dẫn nhất định đối với thực hành giải bài của chúng ta.

Contents

1	Dẫn nhập	2
2	Các khái niệm cơ bản của thuật toán di truyền	2
3	Thuật toán di truyền đơn giản	3
3.1	Chọn lọc	4
3.2	Lai ghép	5
3.3	Đột biến	5
4	Ví dụ tính toán của thuật toán di truyền đơn giản	5
4.1	Ví dụ 3: tối ưu chiến lược kinh doanh của công ty máy tính	5
4.2	Ví dụ 4: bài toán tối ưu hàm	7
5	Ví dụ ứng dụng thuật toán di truyền	8
5.1	Ví dụ 5: ứng dụng GA cho bài toán tổng tập con	8

5.2 Ví dụ 6: ứng dụng GA cho bài toán TSP	10
6 Kết luận	13
A Chương trình nguồn cho bài toán tổng tập con	13
B Chương trình nguồn cho bài toán TSP	17
C Tài liệu tham khảo	21
D Nội dung trang trình chiếu kèm theo	21

1 Dẫn nhập

Thuật toán di truyền là một ngành mới nổi và còn chưa phổ biến trong thi tin học. Tuy nhiên, đối với nhiều bài toán phức tạp mà toán học truyền thống khó giải hoặc tỏ ra kém hiệu quả, đặc biệt là các bài toán tối ưu, thuật toán di truyền cung cấp một con đường mới hữu hiệu. Nó cũng có thể xử lý khá tốt các bài toán khó NP trong thi tin học, vì vậy đáng để ta thảo luận sâu.

Muốn nắm được kỹ thuật ứng dụng thuật toán di truyền, ta phải hiểu các đặc điểm của nó. Trước hết, hãy xem thuật toán di truyền là gì.

2 Các khái niệm cơ bản của thuật toán di truyền

Thuật toán di truyền (Genetic Algorithms, viết tắt là GA) là một nhánh mới quan trọng của trí tuệ nhân tạo. Nó là một ngành được phát triển từ việc mô phỏng cơ chế tiến hóa của sự sống trên máy tính, dựa trên thuyết tiến hóa Darwin. Thuật toán dựa theo các quy luật tiến hóa tự nhiên như thích nghi thì tồn tại, ưu thắng liệt bại để tìm kiếm, tính toán và giải bài toán.

Đối với nhiều bài toán phức tạp mà toán học truyền thống khó giải hoặc tỏ ra không hiệu quả, đặc biệt là các bài toán tối ưu, GA cung cấp một con đường hữu hiệu, đồng thời đem lại sức sống mới cho nghiên cứu trí tuệ nhân tạo.

GA do tiến sĩ J. H. Holland ở Hoa Kỳ đề xuất năm 1975. Khi ấy nó chưa được giới học thuật chú ý, nên phát triển tương đối chậm. Từ giữa thập niên 1980, cùng với sự phát triển của trí tuệ nhân tạo và tiến bộ của công nghệ máy tính, thuật toán di truyền dần trưởng thành và được ứng dụng ngày càng nhiều. Nó không chỉ được dùng trong trí tuệ nhân tạo, chẳng hạn học máy và mạng nơ ron, mà còn bắt đầu được ứng dụng thành công trong các hệ thống công nghiệp như điều khiển, cơ khí, xây dựng dân dụng và kỹ thuật điện, cho thấy triển vọng rất hấp dẫn. Đồng thời, GA cũng được giới học thuật quốc tế thừa nhận rộng rãi.

Từ năm 1985 đến nay, trên thế giới đã tổ chức năm hội nghị về thuật toán di truyền và tính toán tiến hóa. Tạp chí *Evolutionary Computation* đầu tiên được MIT sáng lập năm 1993. Năm 1994, *IEEE Transactions on Neural Networks* xuất bản một chuyên san về lý thuyết và ứng dụng lập trình tiến hóa. Cùng năm đó, IEEE hợp nhất ba hội nghị quốc tế về mạng nơ ron, hệ mờ và tính toán tiến hóa thành Đại hội Trí tuệ Tính toán Toàn cầu IEEE 1994 (WCCI), trong đó có 255 bài báo về tính toán tiến hóa, thu hút sự chú ý rộng rãi của giới học thuật quốc tế.

Hiện nay, GA đã đạt được các kết quả ứng dụng đáng chú ý trong giải bài toán tối ưu tổ hợp, điều khiển thích nghi, sinh chương trình tự động, học máy, huấn luyện mạng nơ ron, nghiên cứu sự sống nhân tạo, tổ hợp kinh tế và nhiều lĩnh vực khác. GA cũng trở thành một chủ đề nóng trong trí tuệ nhân tạo và ứng dụng của nó.

3 Thuật toán di truyền đơn giản

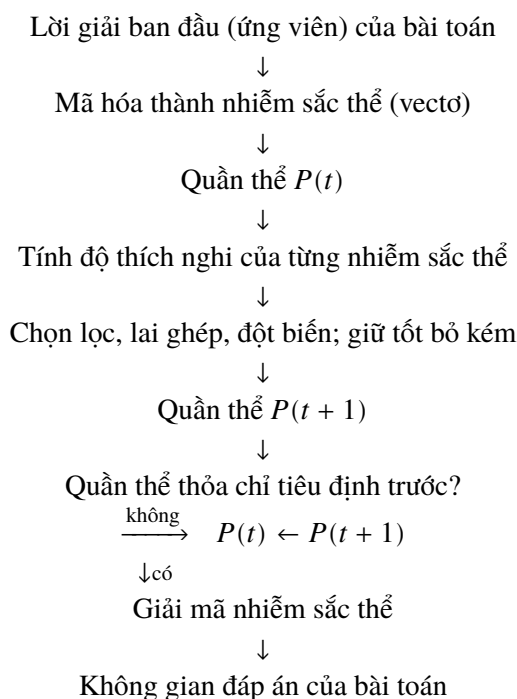
Thuật toán di truyền (Genetic Algorithms, sau đây gọi là GA) là một thuật toán tìm kiếm tối ưu dựa trên chọn lọc tự nhiên và mô phỏng cơ chế tiến hóa sinh học trên máy tính.

Trong quá trình tiến hóa của tự nhiên, sinh vật thích nghi với môi trường bên ngoài thông qua di truyền (truyền giống qua các thế hệ, đời sau rất giống đời trước) và biến dị (đời sau lại không hoàn toàn giống đời trước). Qua từng thế hệ, cá thể tốt được giữ lại, cá thể kém bị loại bỏ, và quần thể phát triển tiến hóa.

GA mô phỏng hiện tượng tiến hóa nói trên. Nó ánh xạ không gian tìm kiếm (không gian lời giải của bài toán cần giải) thành không gian di truyền: mỗi lời giải khả dĩ được mã hóa thành một vectơ, tức một xâu số nhị phân hoặc thập phân, gọi là một **nhiễm sắc thể** (chromosome, hay cá thể). Mỗi phần tử của vectơ gọi là một **gen** (gene). Tất cả nhiễm sắc thể tạo thành **quần thể** (population, hay nhóm). Theo một hàm mục tiêu đã định trước, hoặc một chỉ tiêu đánh giá nào đó như lợi nhuận trong kinh doanh, chi phí nhỏ nhất của công trình, đường đi ngắn nhất, thuật toán đánh giá từng nhiễm sắc thể và cho nó một giá trị **độ thích nghi**.

Khi bắt đầu, thuật toán ngẫu nhiên sinh ra một số nhiễm sắc thể, tức các lời giải ứng viên của bài toán, rồi tính độ thích nghi của chúng. Theo độ thích nghi, thuật toán thực hiện các phép toán di truyền như chọn lọc, lai ghép và đột biến, loại bỏ các nhiễm sắc thể có độ thích nghi thấp (hiệu năng kém) và giữ lại các nhiễm sắc thể có độ thích nghi cao (hiệu năng tốt), từ đó nhận được quần thể mới.

Do các thành viên của quần thể mới là những cá thể ưu tú của thế hệ trước và kế thừa các tính trạng tốt của thế hệ trước, nhìn tổng thể quần thể mới tốt hơn rõ rệt. GA lặp đi lặp lại như vậy, tiến hóa theo hướng lời giải tốt hơn cho đến khi thỏa một chỉ tiêu tối ưu định trước. Quá trình làm việc của GA có thể mô tả bằng sơ đồ sau.



Hình 1. Sơ đồ nguyên lý làm việc của thuật toán di truyền.

Ba phép toán cơ bản của thuật toán di truyền đơn giản là chọn lọc, lai ghép và đột biến. Sau đây trình bày chi tiết.

3.1 Chọn lọc

Phép chọn lọc còn gọi là phép sinh sản, tái sinh hoặc sao chép. Nó dùng để mô phỏng hiện tượng chọn lọc tự nhiên trong sinh giới, tức loại bỏ cái kém và giữ lại cái tốt. Phép này chọn một số nhiễm sắc thể có khả năng thích nghi mạnh từ quần thể cũ và đưa chúng vào tập ghép đôi (vùng đệm), chuẩn bị cho phép lai ghép và đột biến tạo ra quần thể mới. Nhiễm sắc thể có độ thích nghi càng cao thì khả năng được chọn càng lớn, gen của nó càng được phân bố rộng trong quần thể thế hệ sau, và số con cháu xuất hiện trong thế hệ sau càng nhiều. Có nhiều phương pháp chọn lọc; một phương pháp được dùng tương đối phổ biến là phương pháp tỉ lệ độ thích nghi, được mô tả ngắn gọn như sau.

Phương pháp tỉ lệ độ thích nghi còn gọi là phương pháp bánh xe roulette. Nó xem tổng độ thích nghi của mọi nhiễm sắc thể trong quần thể như chu vi của một bánh xe, còn mỗi nhiễm sắc thể chiếm một hình quạt theo tỉ lệ độ thích nghi của nó trong tổng ấy. Mỗi lần chọn một nhiễm sắc thể có thể xem như một lần quay ngẫu nhiên bánh xe; bánh xe dừng ở hình quạt nào thì nhiễm sắc thể tương ứng với hình quạt đó được chọn. Về thực chất, ta xem giá trị độ thích nghi như trọng số: trọng số càng lớn thì xác suất được chọn càng lớn. Tuy phương pháp này có tính ngẫu nhiên, nó vẫn tỉ lệ với độ thích nghi của từng nhiễm sắc thể. Xác suất một nhiễm sắc thể được chọn là

$$P_c = \frac{f(x_c)}{\sum_i f(x_i)}.$$

Trong đó x_i là xâu số tương ứng với nhiễm sắc thể thứ i trong quần thể, $f(x_i)$ là độ thích nghi của nhiễm sắc thể thứ i , và $\sum_i f(x_i)$ là tổng độ thích nghi của mọi nhiễm sắc thể trong quần thể.

Khi chọn theo tỉ lệ độ thích nghi, trước hết tính độ thích nghi của từng nhiễm sắc thể, rồi đưa các nhiễm sắc thể vào tập lai ghép với xác suất tỉ lệ với độ thích nghi của chúng. Các bước cụ thể là:

1. Tính độ thích nghi $f(x_i)$ của từng nhiễm sắc thể.
2. Cộng dồn độ thích nghi của mọi nhiễm sắc thể, nhận giá trị cộng dồn cuối cùng $SUM = \sum_i f(x_i)$, đồng thời ghi lại giá trị cộng dồn trung gian $g(x_i)$ ứng với từng nhiễm sắc thể.
3. Sinh một số ngẫu nhiên N , $0 < N < SUM$.
4. Chọn nhiễm sắc thể có giá trị cộng dồn trung gian thỏa $g(x_{i-1}) < N \leq g(x_i)$ đưa vào tập lai ghép.
5. Lặp lại bước 3 và 4 cho đến khi tập lai ghép chứa đủ số xâu nhiễm sắc thể cần thiết.

Lặp quá trình trên cho đến khi tập lai ghép chứa đủ nhiễm sắc thể. Rõ ràng phương pháp này yêu cầu độ thích nghi của nhiễm sắc thể là số dương. Xem ví dụ sau.

Ví dụ 1. Một quần thể chứa 10 nhiễm sắc thể. Quá trình chọn theo tỉ lệ độ thích nghi để đưa vào tập lai ghép được cho trong Bảng 1 và Bảng 2.

	1	2	3	4	5	6	7	8	9	10
Độ thích nghi	8	2	3	16	6	12	11	7	3	7
Xác suất chọn	0.11	0.03	0.04	0.21	0.08	0.16	0.15	0.09	0.04	0.09
Giá trị cộng dồn	8	10	13	29	35	47	58	65	68	75

Bảng 1.

Số ngẫu nhiên	23	49	70	14	28	37	57
Nhiễm sắc thể được chọn	4	7	10	4	4	6	7

Bảng 2.

Từ Bảng 1 và Bảng 2 có thể thấy nhiễm sắc thể số 4 có xác suất chọn cao nhất và được chọn nhiều lần nhất; các nhiễm sắc thể khác có xác suất chọn thấp hơn thì số lần được chọn ít hơn. Dĩ nhiên, khi chọn theo tỉ lệ độ thích nghi, nhiễm sắc thể kém nhất cũng có thể được chọn, nhưng xác suất cực nhỏ; khi độ dài quần thể lớn, trường hợp này có thể bỏ qua.

3.2 Lai ghép

Phép sao chép tuy có thể chọn ra cá thể ưu tú từ quần thể cũ, nhưng không tạo ra nhiễm sắc thể mới. Vì vậy, người khai sáng thuật toán di truyền đã đề xuất phép lai ghép. Nó mô phỏng hiện tượng sinh sản trong quá trình tiến hóa sinh học. Cụ thể: trong tập ghép đôi, chọn ngẫu nhiên hai nhiễm sắc thể, gọi là hai nhiễm sắc thể cha mẹ; chọn ngẫu nhiên một hoặc nhiều vị trí lai ghép J , với $0 < J < L$, trong đó L là độ dài xâu số nhiễm sắc thể; hoán đổi phần bên phải điểm lai ghép của hai nhiễm sắc thể cha mẹ để nhận được hai xâu nhiễm sắc thể mới của thế hệ sau. Nói cách khác, phép lai ghép có thể tạo ra nhiễm sắc thể mới, tức nhiễm sắc thể con, từ đó cho phép kiểm tra các điểm mới trong không gian tìm kiếm. Lai ghép cũng thể hiện tư tưởng trao đổi thông tin trong tự nhiên. Xem ví dụ sau.

Ví dụ 2. Một cặp nhiễm sắc thể lấy từ tập ghép đôi là:

Nhiễm sắc thể A 0111|1010

Nhiễm sắc thể B 0101|0010

Vị trí lai ghép một điểm được sinh ngẫu nhiên là 4. Hoán đổi phần bên phải vị trí thứ 4 trong nhiễm sắc thể A và B, tức 1010 và 0010, ta nhận được hai xâu nhiễm sắc thể thế hệ sau:

Nhiễm sắc thể A' 01111010

Nhiễm sắc thể B' 01010010

3.3 Đột biến

Phép đột biến dùng để mô phỏng đột biến gen do các nhân tố ngẫu nhiên khác nhau gây ra trong môi trường di truyền của sinh vật. Nó thay đổi ngẫu nhiên giá trị của một gen di truyền, tức một vị trí trong xâu ký hiệu biểu diễn nhiễm sắc thể, với xác suất rất nhỏ. Trong hệ thống dùng mã hóa nhị phân cho nhiễm sắc thể, nó ngẫu nhiên biến một gen nào đó từ 1 thành 0 hoặc từ 0 thành 1. Nếu chỉ có chọn lọc và lai ghép mà không có đột biến, thuật toán sẽ không thể tìm kiếm bên ngoài không gian các tổ hợp gen ban đầu. Quá trình tiến hóa dễ rơi vào lời giải cục bộ ngay từ giai đoạn sớm và dừng lại, khiến chất lượng lời giải bị hạn chế rất lớn. Nhờ phép đột biến, ta bảo đảm tính đa dạng của kiểu gen trong quần thể, để tìm kiếm có thể diễn ra trong không gian rộng nhất có thể, tránh mất thông tin di truyền hữu ích cho tìm kiếm, tránh mắc kẹt ở lời giải cục bộ, và thu được lời giải tối ưu có chất lượng cao hơn.

4 Ví dụ tính toán của thuật toán di truyền đơn giản

Hãy dùng hai ví dụ rất đơn giản sau để hiểu cụ thể các phép toán của thuật toán di truyền đơn giản. Dù các ví dụ này đều có cách xử lý tốt hơn, để hiểu sâu các phép toán của thuật toán di truyền, ta vẫn bắt đầu từ những ví dụ đơn giản.

4.1 Ví dụ 3: tối ưu chiến lược kinh doanh của công ty máy tính

Mục tiêu của một công ty máy tính là lợi nhuận cao nhất. Để đạt mục tiêu đó, công ty phải chọn chiến lược kinh doanh thích hợp. Một chiến lược khả dĩ là đưa ra quyết định cho bốn câu hỏi sau:

- Giá mỗi máy tính PC là 5000 nhân dân tệ (giá thấp) hay 8000 nhân dân tệ (giá cao)?
- Phần mềm miễn phí đi kèm PC là Windows 2000 hay Linux?
- Có cung cấp dịch vụ kỹ thuật mạng hay không?
- Thời hạn bảo hành là nửa năm hay một năm?

Sau đây dùng thuật toán di truyền để giải bài toán tối ưu quyết định này.

1. Biểu diễn các lời giải khả dĩ của bài toán bằng xâu số nhị phân. Vì có bốn biến quyết định, mỗi biến chỉ có hai lựa chọn, giá trị của mỗi biến có thể dùng 0 hoặc 1 để biểu diễn. Do đó, một số nhị phân bốn bit có thể biểu diễn một chiến lược kinh doanh khả dĩ. Không gian tìm kiếm lời giải là $2^4 = 16$, tức có 16 chiến lược kinh doanh để chọn. Bảng 3 cho bốn chiến lược kinh doanh mà giám đốc công ty đã biết, tức các lời giải ban đầu. Trong bảng, bit thứ nhất bằng 0 biểu diễn giá cao, bằng 1 biểu diễn giá thấp; bit thứ hai bằng 0 biểu diễn Windows 2000, bằng 1 biểu diễn Linux đi kèm; bit thứ ba bằng 0 biểu diễn không hỗ trợ dịch vụ kỹ thuật mạng, bằng 1 biểu diễn có hỗ trợ; bit thứ tư bằng 0 biểu diễn bảo hành một năm, bằng 1 biểu diễn bảo hành nửa năm.

STT	Giá	Phần mềm	Dịch vụ mạng	Bảo hành	Xâu nhị phân
1	cao	Linux	hỗ trợ	một năm	0110
2	cao	Windows 2000	hỗ trợ	nửa năm	0011
3	thấp	Linux	không hỗ trợ	một năm	1100
4	cao	Linux	không hỗ trợ	nửa năm	0101

Bảng 3. Lời giải ban đầu của bài toán.

2. Tính độ thích nghi của từng nhị phân. Trong bài toán này, ta có thể giả sử độ thích nghi của nhị phân chính là giá trị của xâu nhị phân, ứng với lợi nhuận của chiến lược kinh doanh.

STT i	Xâu nhị phân x_i	Độ thích nghi $f(x_i)$
1	0110	6
2	0011	3
3	1100	12
4	0101	5
Tổng độ thích nghi		26
Độ thích nghi kém nhất		3
Độ thích nghi tốt nhất		12
Độ thích nghi trung bình		6.5

Bảng 4. Độ thích nghi của quần thể thế hệ 0.

3. Chọn các nhị phân vào tập lai ghép.

Từ Bảng 4, tổng độ thích nghi của mọi xâu nhị phân là 26. Xâu 1100 có độ thích nghi 12, chiếm 6/13 tổng độ thích nghi, nghĩa là xâu 1100 có cơ hội được chọn gần hai lần. Xác suất được chọn của các xâu 0110, 0011, 0101 lần lượt là 3/13, 3/26, 5/26. Theo phương pháp tỉ lệ độ thích nghi đã nói ở trên, các xâu nhị phân được chọn vào tập lai ghép và tình hình độ thích nghi của chúng được cho trong Bảng 5.

STT i	Xâu nhiễm sắc thể x_i	Độ thích nghi $f(x_i)$
1	0110	6
2	1100	12
3	1100	12
4	0101	5
Tổng độ thích nghi		35
Độ thích nghi kém nhất		5
Độ thích nghi tốt nhất		12
Độ thích nghi trung bình		8.75

Bảng 5. Độ thích nghi của quần thể thế hệ 0 sau chọn lọc.

Từ Bảng 5 có thể thấy tác dụng của chọn lọc là cải thiện độ thích nghi trung bình của quần thể, từ 6.5 lên 8.75; độ thích nghi kém nhất cũng từ 3 tăng lên 5, và nhiễm sắc thể có hiệu năng kém nhất đã bị loại khỏi quần thể. Tuy nhiên, chọn lọc không thể tạo ra nhiễm sắc thể mới, nên cần thực hiện lai ghép.

4. Phép lai ghép.

Giả sử dùng lai ghép một điểm, điểm lai ghép sinh ngẫu nhiên là 2. Lấy ngẫu nhiên một cặp nhiễm sắc thể 1100 và 0101 từ tập lai ghép, hoán đổi bit thứ 3 và 4 của chúng, nhận được hai xâu nhiễm sắc thể con 1101 và 0100. Từ đó thu được quần thể thế hệ sau, như Bảng 6.

STT i	Xâu nhiễm sắc thể x_i	Độ thích nghi $f(x_i)$
1	0110	6
2	1101	13
3	1100	12
4	0100	4
Tổng độ thích nghi		35
Độ thích nghi kém nhất		4
Độ thích nghi tốt nhất		13
Độ thích nghi trung bình		8.75

Bảng 6. Độ thích nghi của quần thể thế hệ 1.

5. Đánh giá độ thích nghi của quần thể thế hệ mới.

Từ Bảng 6 có thể thấy trong quần thể mới, tức thế hệ 1, độ thích nghi của nhiễm sắc thể tối ưu đã tăng từ 12 lên 13. Xâu nhị phân tương ứng là 1101, biểu diễn một chiến lược kinh doanh đã được tối ưu. Độ thích nghi trung bình từ 6.5 tăng lên 8.75; nhìn tổng thể độ thích nghi của cả quần thể đã được nâng cao, tức quần thể đã được tối ưu.

Thuật toán di truyền lặp lại các phép toán của từng thế hệ để sinh thế hệ mới cho đến khi thỏa một số tiêu chuẩn hoặc điều kiện dừng. Trong ví dụ này, giả sử đã biết xâu nhiễm sắc thể 1111 biểu diễn lợi nhuận tốt nhất đã biết là 15%, thì thuật toán di truyền dừng chạy.

4.2 Ví dụ 4: bài toán tối ưu hàm

Cho hàm $f(x) = 2x^2 - 5x + 4$, với $x \in \mathbb{Z}$ và $x \in [0, 63]$. Hãy tìm giá trị lớn nhất của nó trong khoảng này. Sau đây dùng thuật toán di truyền để giải.

1. Xác định cách mã hóa thích hợp, biểu diễn các lời giải khả dĩ của bài toán bằng xâu số nhiễm sắc thể. Vì chỉ có một biến quyết định x , phạm vi giá trị là $[0, 63]$, và $2^6 = 64$, dùng xâu nhị phân không dấu 6 bit làm xâu nhiễm sắc thể là đủ để biểu diễn biến x cũng như phương án lời giải của bài toán.

2. Chọn quần thể ban đầu. Dùng phương pháp ngẫu nhiên sinh quần thể ban đầu gồm 4 xâu nhiễm sắc thể, như Bảng 7.
3. Tính độ thích nghi và xác suất chọn. Trong bài toán này, độ thích nghi của nhiễm sắc thể lấy chính hàm $f(x) = 2x^2 - 5x + 4$. Tính $f(x)$ của từng nhiễm sắc thể làm giá trị độ thích nghi. Trên cơ sở đó tính xác suất chọn $P_s = f_i / \sum f$.

Số	Nhiễm sắc thể	X	$f(x) = 2x^2 - 5x + 4$	Xác suất chọn	Số lần chọn
1	011010	26	1226	0.15	1
2	100100	36	2416	0.29	1
3	010101	21	781	0.09	0
4	101110	46	4006	0.48	2
Tổng			8429	1	4
Trung bình			2107.25	0.25	1
Lớn nhất			4006	0.49	2

Bảng 7. Quá trình tối ưu hàm.

4. Chọn nhiễm sắc thể vào tập lai ghép. Theo phương pháp tỉ lệ độ thích nghi, các xâu nhiễm sắc thể được chọn vào tập lai ghép như Bảng 8. Có thể thấy nhiễm sắc thể 1 và 2 đều được chọn 1 lần; nhiễm sắc thể 4 được chọn 2 lần; nhiễm sắc thể 3 không được chọn. Bốn nhiễm sắc thể được chọn được đưa vào tập lai ghép để chuẩn bị lai ghép.
5. Lai ghép nhiễm sắc thể. Trước hết ghép cặp ngẫu nhiên các nhiễm sắc thể trong tập lai ghép. Trong ví dụ này, nhiễm sắc thể 1 ghép với 3, nhiễm sắc thể 2 ghép với 4. Tiếp đó xác định ngẫu nhiên vị trí lai ghép; kết quả là vị trí lai ghép của cặp nhiễm sắc thể thứ nhất là 1, của cặp thứ hai là 4. Sau phép lai ghép, quần thể mới nhận được được cho trong Bảng 8.
6. Đột biến. Nếu xác suất đột biến trong ví dụ này lấy là 0.01, do cả quần thể có 4 nhiễm sắc thể tổng cộng chỉ có 24 bit mã, kỳ vọng số bit đột biến là $24 \cdot 0.01 = 0.24$, có thể bỏ qua.

Tập sau sao chép	Cặp	Vị trí	Nhiễm sắc thể mới	X	$f(x)$
011010	3	3	001110	14	326
100100	4	4	100110	38	2702
101110	1	3	111010	58	6442
101110	2	4	101100	44	3656
Tổng					13126
Trung bình					3281.5
Lớn nhất					6442

Bảng 8. Quá trình tối ưu hàm.

Từ Bảng 8 có thể thấy, tuy chỉ thực hiện một thế hệ phép toán di truyền, giá trị trung bình và giá trị lớn nhất của độ thích nghi quần thể đã tăng rõ rệt so với quần thể ban đầu: trung bình từ 2107.25 thành 3281.5, lớn nhất từ 4006 thành 6442. Điều này cho thấy khi các phép toán di truyền diễn ra, quần thể đang phát triển theo hướng tối ưu.

5 Ví dụ ứng dụng thuật toán di truyền

5.1 Ví dụ 5: ứng dụng GA cho bài toán tổng tập con

Bài toán tổng tập con SUBSET_SUM: cho một tập số nguyên dương S và một số nguyên t , hãy xác định có tồn tại một tập con $S' \subseteq S$ sao cho tổng các số nguyên trong S' bằng t hay không.

Ví dụ, nếu

$$S = \{1, 4, 16, 64, 256, 1040, 1041, 1093, 1284, 1344\}$$

và $t = 3754$, thì tập con

$$S' = \{1, 16, 64, 256, 1040, 1093, 1284\}$$

là một lời giải.

Giả sử một thể hiện của bài toán tổng tập con là $\langle S, t \rangle$, trong đó S là tập các số nguyên dương $\{x_1, x_2, \dots, x_n\}$, và t là một số nguyên dương. Bài toán yêu cầu xác định có tồn tại một tập con S' của S sao cho

$$\sum_{x_i \in S'} x_i = t$$

hay không. Ta đã biết bài toán này là NP-đầy đủ. Trong ứng dụng thực tế, thường gặp dạng tối ưu của bài toán tổng tập con. Khi đó ta cần tìm một tập con S' của S sao cho tổng của nó không vượt quá t , nhưng càng gần t càng tốt. Chẳng hạn, ta có một xe tải với tải trọng không được vượt quá t kg. Có n thùng hàng khác nhau cần dùng xe tải vận chuyển, trong đó thùng thứ i nặng x_i kg. Ta muốn chất xe tải đầy nhất có thể mà không vượt quá giới hạn tải trọng. Bài toán này về bản chất chính là dạng tối ưu của bài toán tổng tập con.

Sau đây dùng thuật toán di truyền để giải.

Nếu số phần tử trong tập S là n , mỗi phần tử chỉ có hai khả năng: thuộc S' hoặc không thuộc S' . Vì vậy ta có thể dùng một số nhị phân n bit để biểu diễn mỗi nhiễm sắc thể. Với từng bit, nếu bằng 0 thì phần tử tương ứng không thuộc S' , nếu bằng 1 thì phần tử tương ứng thuộc S' .

Sau khi xác định cách mô tả bài toán, ta chỉ cần lấy ngẫu nhiên các nhiễm sắc thể để tạo quần thể ban đầu. Tiếp đó có thể thực hiện chọn lọc, lai ghép và đột biến như đã nói ở trên.

Ta ghi độ chênh lệch giữa tổng các phần tử của tập con do nhiễm sắc thể biểu diễn và t cho trước làm độ thích nghi. Cụ thể, gọi bit thứ i của nhiễm sắc thể x là x_i , giá trị phần tử tương ứng là S_i , thì

$$f(x) = \sum_{x_i \neq 0} S_i - t.$$

Tuy nhiên sau thực nghiệm, do chênh lệch tương đối giữa các độ thích nghi nhỏ, các giá trị độ thích nghi rất gần nhau, khó phân biệt nhiễm sắc thể tốt, khiến quá trình tiến hóa di truyền rất chậm; hơn nữa $f(x)$ có thể là số âm. Vì vậy cần biến đổi hàm độ thích nghi để phù hợp với yêu cầu bài toán.

Gọi $|f(k)|$ là giá trị lớn nhất của độ thích nghi của mọi nhiễm sắc thể trong quần thể hiện tại. Đặt

$$f'(x) = |f(k) - f(x)|.$$

Khi đó độ thích nghi dùng là $f'(x)$.

Khi chọn lọc, có thể dùng phương pháp tỉ lệ độ thích nghi đã giới thiệu ở trên. Nhưng nếu dùng phương thức ngẫu nhiên, có thể xuất hiện sai lệch ngẫu nhiên làm cho nhiễm sắc thể ưu tú không có con cháu. Vì vậy ở đây ta cải tiến phương pháp chọn lọc, dùng phương pháp chọn lọc xác định để lựa chọn không phụ thuộc vào may rủi của ngẫu nhiên. Trước hết tính xác suất sống sót của từng xâu trong quần thể:

$$P_s = \frac{f_i}{\sum_j f_j}, \quad 1 \leq j \leq n.$$

Sau đó tính số lần sao chép kỳ vọng $e_i = P_s \cdot n$, trong đó n là số nhiễm sắc thể trong quần thể. Theo phần nguyên của e_i , phân phối một số lần sao chép cho từng xâu nhiễm sắc thể; rồi sắp xếp các xâu nhiễm sắc thể trong quần thể theo phần thập phân của e_i . Cuối cùng, theo thứ tự sắp xếp này, chọn các xâu nhiễm sắc thể, tức con cháu, được giữ lại cho thế hệ sau.

Tiếp đó có thể thực hiện phép lai ghép. Phép lai ghép giống như phần trước. Tuy nhiên nếu dùng lai ghép một điểm, hai nhiễm sắc thể có thể sinh ra khác biệt quá lớn khi lai ghép, làm tiến hóa không ổn định và nhiễm sắc thể ưu tú không truyền được sang thế hệ sau. Vì vậy có thể dùng lai ghép nhiều điểm. Trong bài này chỉ cần lai ghép hai điểm; thực ra lai ghép hai điểm tương tự lai ghép một điểm.

Giả sử có hai nhiễm sắc thể cha mẹ A và B:

A: 10100100101110

B: 01001110110001

Giả sử hai điểm lai ghép được chọn như sau:

A: 10100|10010|1110

B: 01001|11011|0001

Phép lai ghép hai điểm là hoán đổi phần nằm giữa hai điểm lai ghép của nhiễm sắc thể A và nhiễm sắc thể B. Kết quả lai ghép là:

A' : 10100110111110

B' : 01001100100001

Sau khi hoàn thành lai ghép, có thể tiến hành đột biến. Khi đột biến, chỉ cần chú ý giá trị của xác suất đột biến; thuật toán cụ thể như đã nói ở trên.

Trong bài này, các giá trị có thể lấy như sau:

- Độ dài quần thể (số nhiễm sắc thể): 20.
- Xác suất chọn: 0.9.
- Xác suất đột biến: 0.1.
- Điều kiện dừng: lời giải tốt nhất hiện tại không thay đổi sau 100 thế hệ di truyền, hoặc đã đạt lời giải tối ưu.

5.2 Ví dụ 6: ứng dụng GA cho bài toán TSP

Giả sử có N thành phố, D_{ij} biểu diễn khoảng cách giữa thành phố i và thành phố j , với $D_{ij} = D_{ji}$. Nay yêu cầu một đường đi ngắn nhất đi qua toàn bộ N thành phố và không đi lặp đường, tức chu trình Hamilton ngắn nhất.

Đây là một bài toán điển hình về thứ tự hoán vị, thuộc lớp NP-đầy đủ. Các cách giải truyền thống như nhánh cận hoặc thuật toán cây khung nhỏ nhất đều không mấy hiệu quả ở đây, hoặc do hạn chế tốc độ tính toán, hoặc do hạn chế chất lượng lời giải. Sau đây ta thử dùng thuật toán di truyền để giải bài này.

Trước hết dùng mã hóa thập phân. Mỗi nhiễm sắc thể gồm dãy số hiệu của N thành phố theo một thứ tự nào đó, biểu diễn một hành trình khả dĩ; mỗi gen biểu diễn một thành phố. Độ dài nhiễm sắc thể là N , không gian lời giải là $N!$. Độ thích nghi là khoảng cách tương ứng với một hành trình; hành trình càng ngắn thì nhiễm sắc thể càng thích nghi cao. Ví dụ, lấy $N = 10$, mã thành phố là 1, 2, 3, 4, 5, 6, 7, 8, 9, 10.

Nhiệm sắc thể trong quan thế: 2 8 4 10 5 1 7 3 6 9

Hành trình tương ứng: 2 -> 8 -> 4 -> 10 -> 5 -> 1 -> 7 -> 3 -> 6 -> 9 -> 2

Tổng độ dài đường đi là

$$\sum D_{ij} = D_{28} + D_{84} + D_{4,10} + D_{10,5} + D_{51} + D_{17} + D_{73} + D_{36} + D_{69} + D_{92}.$$

Ta có thể dùng phép biến đổi không âm để đổi hàm mục tiêu tối thiểu hóa thành hàm độ thích nghi lấy giá trị lớn làm mục tiêu. Có thể định nghĩa:

$$f(x) = c_{\max} - \sum D_{ij}.$$

Trong đó $c_{\max}$ có thể lấy là độ dài đường đi lớn nhất trong quá trình tiến hóa, hoặc để bảo đảm $f(x)$ dương thì đặt trước một hằng số độc lập với quần thể.

Sau đây giới thiệu thuật toán cụ thể.

Trước hết, từ số thành phố N , ta xác định được độ dài nhiễm sắc thể là N , và mã gen của nhiễm sắc thể do mã thành phố tạo thành. Tuy nhiên, ta vẫn phải xác định các tham số như độ dài quần thể, xác suất đột biến. Các tham số này đều do con người xác định, không phụ thuộc vào bản thân thuật toán; tạm thời có thể không xét, sau khi thảo luận xong thuật toán sẽ xác định giá trị cụ thể.

Tạo ngẫu nhiên các tổ hợp thành nhiễm sắc thể; mỗi nhiễm sắc thể phải biểu diễn một chu trình Hamilton, từ đó sinh quần thể ban đầu. Sau đó tính riêng độ thích nghi của từng nhiễm sắc thể và chọn nhiễm sắc thể vào tập lai ghép theo phương pháp tỉ lệ độ thích nghi. Tuy nhiên ta cũng có thể cân nhắc phương pháp chọn lọc đã cải tiến.

Phương pháp tỉ lệ độ thích nghi cài đặt đơn giản, nhưng có nhược điểm là trong quá trình chọn, nhiễm sắc thể tốt nhất có thể không sinh được con cháu ở thế hệ sau, tức có thể xảy ra sai lệch ngẫu nhiên. Vì vậy có thể dùng phương pháp chọn ưu tú: các nhiễm sắc thể tốt nhất trong quần thể, tức có độ thích nghi cao nhất, không tham gia lai ghép và đột biến mà được sao chép trực tiếp sang thế hệ sau, tránh để phép lai ghép và đột biến phá hỏng các lời giải tốt trong quần thể. Phương pháp này có thể tăng tốc tìm kiếm cục bộ, nhưng cũng có thể làm tìm kiếm rơi vào tối ưu cục bộ do số nhiễm sắc thể tốt trong quần thể tăng quá nhanh. Đồng thời, ta cũng có thể dùng phương pháp chọn lọc xác định để việc chọn không phụ thuộc vào may rủi ngẫu nhiên. Trước hết tính xác suất sống sót của từng xâu trong quần thể:

$$P_s = \frac{f_i}{\sum_j f_j}, \quad 1 < j < n.$$

Sau đó tính số lần sao chép kỳ vọng $e_i = P_s \cdot n$, trong đó n là số nhiễm sắc thể trong quần thể. Theo phần nguyên của e_i , phân phối số lần sao chép cho từng xâu nhiễm sắc thể; theo phần thập phân của e_i , sắp xếp các xâu nhiễm sắc thể trong quần thể; cuối cùng theo thứ tự sắp xếp này, chọn các xâu nhiễm sắc thể, tức con cháu, được giữ lại cho thế hệ sau.

Sau khi hoàn thành chọn lọc, có thể thực hiện lai ghép. Nhưng phép lai ghép ở đây khác với trước, cần cải tiến phép lai ghép ban đầu. Cách làm cụ thể như sau.

Nếu thực hiện phép lai ghép đơn giản trên hai nhiễm sắc thể, đường đi mà nhiễm sắc thể biểu diễn có thể đi qua cùng một thành phố nhiều lần, nghĩa là hai gen trong cùng một nhiễm sắc thể có cùng mã thành phố. Khi đó nhiễm sắc thể không còn biểu diễn một chu trình Hamilton, vì vậy cần cải tiến phép lai ghép.

Ta có thể dùng phép lai ghép khớp bộ phận (partially matched crossover, PMX). Phép này trước hết dùng phân bố đều ngẫu nhiên để sinh hai điểm lai ghép trên mỗi xâu nhiễm sắc thể cha mẹ cần lai ghép, định nghĩa vùng giữa hai điểm ấy là vùng khớp, rồi hoán vị các gen trong hai vùng khớp theo quan hệ khớp tương ứng. Ví dụ, hai xâu nhiễm sắc thể cha mẹ là:

A: 2 8 4 10 * 5 1 7 3 * 6 9
B: 5 6 7 1 * 10 2 8 3 * 9 4

Ký hiệu * biểu diễn điểm lai ghép.

Với xâu nhiễm sắc thể A, vùng khớp có bốn phần tử 5, 1, 7, 3, cần khớp lần lượt với bốn phần tử 10, 2, 8, 3 trong vùng khớp của xâu nhiễm sắc thể B. Nói cách khác, bằng cách hoán vị các phần tử bên

trong xâu nhiễm sắc thể A, ta làm cho bốn phần tử trong vùng khớp đổi thành 10, 2, 8, 3. Vì vậy các phần tử trong xâu nhiễm sắc thể A được hoán vị như sau:

A : 2 8 4 10 * 5 1 7 3 * 6 9
A' : 1 7 4 5 * 10 2 8 3 * 6 9

Với xâu nhiễm sắc thể B, vùng khớp có bốn phần tử 10, 2, 8, 3, cần khớp lần lượt với bốn phần tử 5, 1, 7, 3 trong vùng khớp của xâu nhiễm sắc thể A. Bằng cách hoán vị các phần tử bên trong xâu nhiễm sắc thể B, ta làm cho bốn phần tử trong vùng khớp đổi thành 5, 1, 7, 3. Vì vậy các phần tử trong xâu nhiễm sắc thể B được hoán vị như sau:

B : 5 6 7 1 * 10 2 8 3 * 9 4
B' : 10 6 8 2 * 5 1 7 3 * 9 4

Dĩ nhiên, trong bài này cách đơn giản nhất là không thực hiện lai ghép mà trực tiếp đột biến, vì phép đột biến ở đây vốn đã hàm chứa ý nghĩa hoán đổi.

Đối với một số nhiễm sắc thể trong quần thể, ta thực hiện đột biến. Do gen dùng mã thành phố, phép đột biến khác với khi mã hóa nhị phân: lấy ngẫu nhiên một nhiễm sắc thể từ quần thể, lấy ngẫu nhiên hai gen, rồi hoán đổi chúng, tức đảo thứ tự hai thành phố, để hoàn thành đột biến.

Dĩ nhiên ta còn cần xác định điều kiện dừng. Hiện nay vẫn khá khó dùng phương pháp toán học nghiêm ngặt để phán định điều kiện dừng, tức hội tụ, của thuật toán di truyền. Vì vậy khi cài đặt, chủ yếu vẫn dùng phương pháp heuristic: có thể lấy việc lời giải tốt nhất trong nhóm lời giải sau vài lần lặp liên tiếp có thay đổi hay không làm điều kiện phán định khi dừng.

Phần chính của thuật toán đã thảo luận xong, nhưng còn một điểm đáng nêu. Thuật toán di truyền là một thuật toán tìm kiếm tối ưu dần dần, vì vậy chất lượng của quần thể ban đầu sẽ ảnh hưởng đến chất lượng lời giải cuối cùng và tốc độ tìm ra lời giải. Do đó ta có thể dùng thuật toán tham lam để xây dựng quần thể ban đầu; hoặc khi bài toán TSP có tính chất bất đẳng thức tam giác, ta có thể cân nhắc dùng lời giải của thuật toán cây khung nhỏ nhất, vốn có tốc độ nhanh, để xây dựng quần thể ban đầu.

Dùng thuật toán cây khung nhỏ nhất để xây dựng quần thể ban đầu có cả lợi và hại. Nhờ quần thể ban đầu có chất lượng cao, thuật toán di truyền có thể hội tụ trong thời gian ngắn hơn; lời giải thu được có bảo đảm hơn về chất lượng và độ ổn định cũng cao. Nhưng vì quần thể ban đầu quá tốt nên ảnh hưởng của nó đến đời sau rất lớn, dễ gây hội tụ sớm vào tối ưu cục bộ. Vì vậy khi sử dụng, cần quyết định tùy tình hình, cân nhắc tổng hợp yêu cầu về chất lượng lời giải và yêu cầu về thời gian. Đây là một số kinh nghiệm tôi tổng kết khi giải bài toán TSP, hy vọng có thể có ý nghĩa hướng dẫn nhất định cho mọi người trong quá trình giải bài.

Trong bài toán này, các giá trị có thể lấy như sau:

- Độ dài quần thể (số nhiễm sắc thể): 20.
- Xác suất đột biến: 0.1.

Qua hai ví dụ, trên định nghĩa đơn giản ban đầu của thuật toán di truyền, ta đã mở rộng thêm và giới thiệu một số ứng dụng của thuật toán di truyền bậc cao trong các trường hợp cụ thể. Mục đích là mở rộng giới hạn tư duy: không chỉ làm việc trong định nghĩa đơn giản ban đầu, mà phải phát huy đầy đủ trí tưởng tượng, với các bài toán khác nhau thì dùng đối sách khác nhau. Trong khuôn khổ thuật toán di truyền, ta cần bố trí và sử dụng thuật toán hợp lý, cải tiến thuật toán ban đầu, hoặc kết hợp với các thuật toán sẵn có. Chỉ như vậy mới phát huy đầy đủ ưu điểm của thuật toán di truyền để giải bài toán.

6 Kết luận

Nguyên lý của thuật toán di truyền là đơn giản, nhưng dùng thành thạo thuật toán di truyền lại không phải là một vấn đề đơn giản. Lý thuyết phải kết hợp với thực tế; đối với các bài toán khác nhau, thuật toán di truyền cần thay đổi một chút, giống như nét chấm mắt làm rồng sống dậy. Tuyệt đối không nên áp dụng máy móc. Tôi cho rằng thuật toán di truyền nên kết hợp với các thuật toán tối ưu hiện có; như vậy có thể sinh ra các thuật toán tốt hơn và thực dụng hơn so với chỉ dùng riêng thuật toán di truyền hoặc chỉ dùng riêng thuật toán tối ưu hiện có.

Mục đích chính của bài viết này vẫn là giúp mọi người có hiểu biết ban đầu về thuật toán di truyền, từ đó có ích cho việc thực hành sâu hơn. Hy vọng qua sự hướng dẫn của bài viết này, mọi người có thể ứng dụng thành thạo thuật toán di truyền trong nhiều phương diện.

A Chương trình nguồn cho bài toán tổng tập con

```
program subset_sum_GA;
const inp='subset.in';{tệp vào}
      out='subset.out';{tệp ra}
      maxn=1000;{số phần tử lớn nhất của tập}
      pl=20;{số nhiệm vụ các thế trong quần thể}
      pc=10;{số lần lai ghép}
      pv=0.1;{xác suất đột biến}
      ec=100;{điều kiện dừng}
type ch=array[1..maxn] of 0..1;
var sset:array[1..maxn] of integer;{tập}
    pop, popl:array[1..pl] of ch;{quần thể}
    result:ch;{kết quả}
    n:integer;{số phần tử của tập}
    t{tổng đích},sum{tổng tập},diff{chênh lệch tốt nhất}:longint;
procedure input;{nhập}
var fin:text;
    i:integer;
begin
  assign(fin,inp);
  reset(fin);
  readln(fin,n,t);
  for i:=1 to n do
    readln(fin,sset[i]);
  close(fin);
end;
procedure init;{khởi tạo}
var i,j,k:integer;
begin
  randomize;
  sum:=0;
  for i:=1 to n do
    inc(sum,sset[i]);
  if sum<=t then
```

```

begin
  for i:=1 to n do
    result[i]:=1;
  exit;
end;
k:=trunc(sum/t)+1;
for i:=1 to pl do
  begin
    for j:=1 to n do
      if random(k)=0 then
        pop[i,j]:=1
      else
        pop[i,j]:=0;
    end;
  diff:=t;
end;
procedure solve;{thuat toan di truyen}
var f1:array[1..pl] of longint;{do thich nghi}
    dc,pr:integer;
    work:boolean;
procedure select;{phep chon loc}
var f:array[1..pl] of longint;{do thich nghi}
    g:array[1..pl] of integer;{so lan sao chep}
    fs,fm:longint;
    i,j,k:integer;
begin
  fm:=0;
  for i:=1 to pl do
    begin{tinh do thich nghi}
      f[i]:=0;
      for j:=1 to n do
        inc(f[i],sset[j]*pop[i,j]);
      f[i]:=f[i]-t;
      if (f[i]<0) and (abs(f[i])<diff) then
        begin
          diff:=abs(f[i]);
          dc:=0;
          result:=pop[i];
        end;
      if f[i]=0 then
        begin
          work:=false;
          exit;
        end;
      if abs(f[i])<abs(fm) then
        fs:=f[i];
    end;
  inc(dc);

```

```

for i:=1 to pl do{bien doi do thich nghi}
  if f[i]<0 then
    f[i]:=fm+f[i]
  else
    f[i]:=fm-f[i];
fs:=0;
for i:=1 to pl do{tinh tong do thich nghi}
  inc(fs,abs(f[i]));
k:=0;
for i:=1 to pl do
  begin{phan phoi so lan sao chep}
    g[i]:=trunc(abs(f[i])/fs*pl);
    inc(k,g[i]);
  end;
while k<pl do
  begin
    inc(g[random(pl)+1]);
    inc(k);
  end;
k:=0;
for i:=1 to pl do{chep vao tap lai ghep}
  for j:=1 to g[i] do
    begin
      inc(k);
      pop1[k]:=pop[i];
      fl[k]:=f[i];
    end;
  pop:=pop1;
end;
procedure cross;{phep lai ghep}
var i,j,k,u,v,p,q:integer;
begin
  for i:=1 to pc do
    begin
      u:=random(pl)+1;{nhien sac the A}
      v:=random(pl)+1;{nhien sac the B}
      p:=random(n)+1;{diem lai ghep 1}
      q:=random(n)+1;{diem lai ghep 2}
      if p>q then
        begin
          k:=p;
          p:=q;
          q:=k;
        end;
      k:=0;
      for j:=p to q do
        inc(k,pop[u,j]-pop[v,j]);{tinh chenh lech phan lai ghep}
      if not (((fl[u]<0) and (fl[v]>0) and (k>0)) or

```

```

        ((f1[u]>0) and (f1[v]<0) and (k<0))) then{neu xau hon thi bo}
        continue;
    dec(f1[u],k);
    inc(f1[v],k);
    for j:=p to q do
        begin{lai ghep}
            k:=pop[u,j];
            pop[u,j]:=pop[v,j];
            pop[v,j]:=k;
        end;
    end;
end;
procedure variety;{phep dot bien}
var i,j,k:integer;
begin
    for i:=1 to pr do
        begin
            j:=random(pl)+1;{nhien sac the}
            k:=random(n)+1;{gen}
            if ((f1[j]<0) and (pop[j,k]=0)) or {dot bien}
                ((f1[j]>0) and (pop[j,k]=1)) then
                begin
                    inc(f1[j],(1-2*pop[j,k])*sset[k]);
                    pop[j,k]:=1-pop[j,k];
                end;
        end;
    end;
begin
    if sum<=t then
        exit;
    work:=true;
    pr:=round(n*pl*pv);{tinh so lan dot bien}
    dc:=0;
    while work do
        begin
            select;{chon loc}
            if dc>ec then
                exit;{kiem tra dieu kien dung}
            if work then
                begin
                    cross;{lai ghep}
                    variety;{dot bien}
                end;
        end;
    end;
end;
procedure output;{xuat}
var fout:text;
    i,k:integer;

```



```

        s:longint;
begin
    assign(fout,out);
    rewrite(fout);
    s:=0;
    k:=0;
    for i:=1 to maxn do
        if result[i]=1 then
            begin
                inc(s,sset[i]);
                inc(k);
            end;
    writeln(fout,s);
    writeln(fout,k);
    for i:=1 to maxn do
        if result[i]=1 then
            writeln(fout,i);
    close(fout);
end;
begin{chuong trinh chinh}
input;
init;
solve;
output;
end.

```

Giải thích. Trong bài toán tổng tập con, với dữ liệu được sinh ngẫu nhiên, về cơ bản đều có thể đạt lời giải tối ưu; thỉnh thoảng cũng hội tụ tới lời giải gần tối ưu. Tình hình tổng thể rất tốt.

B Chương trình nguồn cho bài toán TSP

```

program TSP_GA;
const inp='tsp.in';{tep vao}
      out='tsp.out'; {tep ra}
      maxn=100;{gia tri lon nhat cua so thanh pho}
      pl=100;{so nhien sac the}
      pv=0.1;{xac suat di truyen}
      ec=100;{dieu kien dung}
      no=10000;{khong co duong di}
type ch=array[1..maxn] of integer;
var d:array[1..maxn,1..maxn] of integer;{ma tran ke}
    pop,popl:array[1..pl] of ch;{nhien sac the}
    result:ch;{ket qua}
    n:integer;{so thanh pho}
    min:longint;{do dai duong di ngan nhat hien tai}
procedure input;{nhap}
var i,j:integer;

```

```

    fin:text;
begin
    assign(fin,inp);
    reset(fin);
    readln(fin,n);
    for i:=1 to n do
        for j:=1 to n do
            begin
                read(fin,d[i,j]);
                if d[i,j]=-1 then
                    d[i,j]:=no;
            end;
        end;
    end;
    close(fin);
end;
procedure init;{khởi tạo}
var temp:ch;
    i:integer;
procedure greedy;{tham lam}
var use:array[1..maxn] of boolean;
    i,j,k,min,r:integer;
begin
    r:=random(n)+1;
    fillchar(use,sizeof(use),true);
    temp[1]:=r;
    use[r]:=false;
    for i:=2 to n do
        begin
            min:=no+1;
            k:=0;
            for j:=1 to n do
                if use[j] then
                    if d[r,j]<min then
                        begin
                            min:=d[r,j];
                            k:=j;
                        end;
            end;
            temp[i]:=k;
            use[k]:=false;
            r:=k;
        end;
    end;
begin{khởi tạo quần thể ban đầu}
    randomize;
    for i:=1 to pl do
        begin
            greedy;
            pop[i]:=temp;
        end;
end;

```

```

    min:=no;
end;
procedure solve;{thuat toan di truyen}
var f1:array[1..pl] of longint;
    dc,pr:integer;
    work:boolean;
function suit(a:ch):longint;{do thich nghi}
var p:longint;
    i:integer;
begin
    p:=0;
    for i:=1 to n-1 do
        inc(p,d[a[i],a[i+1]]);
    inc(p,d[a[n],a[1]]);
    suit:=p;
end;
procedure select;{chon loc}
var f:array[1..pl] of longint;
    g:array[1..pl] of integer;
    fs,fm:longint;
    i,j,k:integer;
begin
    fm:=0;
    for i:=1 to pl do
        begin
            f[i]:=suit(pop[i]);
            if f[i]<min then
                begin
                    min:=f[i];
                    result:=pop[i];
                    dc:=0;
                end;
            if f[i]>fm then
                fm:=f[i];
        end;
    inc(dc);
    fs:=0;
    for i:=1 to pl do
        inc(fs,fm+1-f[i]);
    for i:=1 to pl do
        g[i]:=trunc((fm+1-f[i])/fs*pl);
    k:=0;
    for i:=1 to pl do
        for j:=1 to g[i] do
            begin
                inc(k);
                pop1[k]:=pop[i];
                f1[k]:=f[i];
            end;
        end;
    end;
end;

```

```

        end;
    for i:=k+1 to pl do
    begin
        pop1[i]:=result;
        fl[i]:=min;
    end;
    pop:=pop1;
end;
procedure variety;{dot bien}
var i,j,k,l,p:integer;
    t:longint;
    temp:ch;
begin
    for i:=1 to pr do
    begin
        j:=random(pl)+1;
        k:=random(n)+1;
        l:=random(n)+1;
        temp:=pop[j];
        p:=temp[k];
        temp[k]:=temp[l];
        temp[l]:=p;
        t:=suit(temp);
        if t<fl[j] then
        begin
            pop[j]:=temp;
            fl[j]:=t;
        end;
    end;
end;
begin
    work:=true;
    pr:=round(n*pl*pv);
    dc:=0;
    while true do
    begin
        select;
        if dc>ec then
            exit;
        if work then
            variety;
    end;
end;
procedure output;{xuat}
var i:integer;
    fout:text;
begin
    assign(fout,out);

```

```

        rewrite(fout);
    for i:=1 to n do
        writeln(fout,result[i]);
    close(fout);
end;
begin{chương trình chính}
    input;
    init;
    solve;
    output;
end.

```

Giải thích. Vì bản thân bài toán TSP là một bài toán NP, với dữ liệu được sinh ngẫu nhiên ta không có cách biết lời giải tối ưu. Hơn nữa, do tính ngẫu nhiên, khoảng cách giữa lời giải tối ưu và lời giải tối ưu cục bộ thường không quá rõ, dù các tuyến đường có thể hoàn toàn khác nhau. Vì vậy thuật toán có thể hội tụ tới lời giải tối ưu cục bộ. Còn với dữ liệu do con người xây dựng, khi lời giải tối ưu có khác biệt tương đối rõ, nói chung thuật toán có thể hội tụ tới lời giải tối ưu.

C Tài liệu tham khảo

1. Shao Junli, Zhang Jing, Wei Changhua, *Cơ sở trí tuệ nhân tạo*, Nhà xuất bản Công nghiệp Điện tử.
2. Fu Qingxiang, Wang Xiaodong, *Thuật toán và cấu trúc dữ liệu*, Nhà xuất bản Công nghiệp Điện tử.
3. Wu Wenhui, Wang Jiande, *Phân tích và lập trình thuật toán thực dụng*, Nhà xuất bản Công nghiệp Điện tử.

D Nội dung trang trình chiếu kèm theo

Bản trích xuất từ PDF còn có một nhóm trang trình chiếu gắn đi kèm sau phần tài liệu tham khảo. Nội dung chính của các trang này như sau.

Trang bìa. Đặc điểm và ứng dụng của thuật toán di truyền. Tỉnh, thành phố: Thượng Hải. Trường: Trung học trực thuộc Đại học Phúc Đán. Họ tên: Zhang Ning.

Mục lục.

- Các khái niệm cơ bản của thuật toán di truyền.
- Thuật toán di truyền đơn giản: chọn lọc, lai ghép, đột biến.
- Ví dụ ứng dụng thuật toán di truyền: bài toán tổng tập con và bài toán TSP (người bán hàng du lịch).
- Kết luận.

Khái niệm cơ bản. Thuật toán di truyền (Genetic Algorithms, viết tắt là GA) tiến hành tìm kiếm, tính toán và giải bài toán theo các quy luật tiến hóa tự nhiên như thích nghi thì tồn tại, ưu thắng liệt bại. Đối

với nhiều bài toán phức tạp mà toán học truyền thống khó giải hoặc rõ ràng kém hiệu quả, đặc biệt là các bài toán tối ưu, GA cung cấp một con đường hữu hiệu.

Thuật toán di truyền đơn giản. GA mã hóa mỗi lời giải khả dĩ thành một vectơ gọi là nhiễm sắc thể; mỗi phần tử của vectơ gọi là gen. Tất cả nhiễm sắc thể tạo thành quần thể. Theo hàm mục tiêu đã định, thuật toán đánh giá từng nhiễm sắc thể và cho nó một giá trị độ thích nghi.

Khi bắt đầu, thuật toán sinh ngẫu nhiên một số nhiễm sắc thể, tính độ thích nghi của chúng, rồi theo độ thích nghi thực hiện chọn lọc, lai ghép, đột biến và các phép toán di truyền khác. Các nhiễm sắc thể có độ thích nghi thấp bị loại bỏ; các nhiễm sắc thể có độ thích nghi cao được giữ lại.

Vì các thành viên của quần thể mới là cá thể ưu tú của thế hệ trước, nhìn tổng thể chúng tốt hơn thế hệ trước. GA lặp lại như vậy cho đến khi thỏa một chỉ tiêu tối ưu định trước. Trang trình chiếu này lặp lại Hình 1 về nguyên lý làm việc của thuật toán di truyền, đã được dựng lại trong phần chính.

Chọn lọc. Một phương pháp chọn lọc được dùng khá phổ biến là phương pháp tỉ lệ độ thích nghi. Nó xem giá trị độ thích nghi là trọng số; trọng số càng lớn thì xác suất được chọn càng lớn. Xác suất này tỉ lệ với độ thích nghi của từng nhiễm sắc thể:

$$P_c = \frac{f(x_c)}{\sum_i f(x_i)}.$$

Trong đó x_i là xâu số tương ứng với nhiễm sắc thể thứ i trong quần thể, $f(x_i)$ là độ thích nghi của nhiễm sắc thể thứ i , và $\sum_i f(x_i)$ là tổng độ thích nghi của mọi nhiễm sắc thể trong quần thể. Phương pháp này yêu cầu độ thích nghi của nhiễm sắc thể là số dương.

Lai ghép. Phép sao chép tuy có thể chọn ra cá thể ưu tú từ quần thể cũ, nhưng không tạo ra nhiễm sắc thể mới. Vì vậy thuật toán di truyền dùng phép lai ghép: trong tập ghép đôi, chọn tùy ý hai nhiễm sắc thể, chọn ngẫu nhiên một hoặc nhiều vị trí lai ghép, rồi hoán đổi phần bên phải điểm lai ghép của hai nhiễm sắc thể cha mẹ để nhận được hai xâu nhiễm sắc thể mới.

Đột biến. Phép đột biến mô phỏng đột biến gen do các nhân tố ngẫu nhiên trong môi trường di truyền của sinh vật gây ra. Nó thay đổi ngẫu nhiên giá trị gen với xác suất rất nhỏ. Trong hệ thống mã hóa nhị phân, nó ngẫu nhiên đổi một gen từ 1 thành 0 hoặc từ 0 thành 1. Nhờ đột biến, tìm kiếm có thể diễn ra trong không gian rộng nhất có thể và thu được lời giải tối ưu chất lượng cao hơn.

Bài toán tổng tập con. Bài toán SUBSET_SUM: cho tập số nguyên dương S và số nguyên t , hãy xác định có tồn tại một tập con S' sao cho tổng các số trong S' bằng t hay không. Đây là một bài toán NP-đầy đủ. Trong thực tế, thường gặp dạng tối ưu: tìm tập con S' sao cho tổng của nó không vượt quá t , nhưng càng gần t càng tốt.

Ta dùng một số nhị phân n bit để biểu diễn mỗi nhiễm sắc thể; từng bit 0 hoặc 1 biểu diễn phần tử tương ứng có thuộc tập con hay không. Độ chênh lệch giữa tổng phần tử của tập con do nhiễm sắc thể biểu diễn và t cho trước được ghi làm độ thích nghi:

$$f(x) = \sum_{x_i \neq 0} S_i - t.$$

Do các độ thích nghi tương đối gần nhau, khó phân biệt ưu khuyết của nhiễm sắc thể, và $f(x)$ có thể âm, cần biến đổi hàm độ thích nghi:

$$f'(x) = |f(k) - f(x)|.$$

Trong đó $f(k)$ là độ thích nghi lớn nhất trong quần thể hiện tại.

Khi chọn lọc, có thể dùng phương pháp tỉ lệ độ thích nghi, nhưng để tránh trường hợp cá thể ưu tú không có con cháu do ngẫu nhiên, trang trình chiếu dùng phương pháp chọn lọc xác định. Trước hết tính xác suất sống sót của từng cá thể:

$$P_s = \frac{f_i}{\sum_j f_j}, \quad 1 \leq j \leq n.$$

Sau đó tính số lần sao chép kỳ vọng $e_i = P_s \cdot n$, trong đó n là số nhiễm sắc thể trong quần thể, rồi theo giá trị e_i phân phối số lần sao chép cho từng cá thể nhiễm sắc thể.

Phép lai ghép có thể dùng nhiều điểm vì lai ghép một điểm có thể làm khác biệt giữa hai nhiễm sắc thể quá lớn, khiến tiến hóa không ổn định và cá thể ưu tú không truyền được sang thế hệ sau. Phép đột biến chỉ cần chú ý giá trị xác suất đột biến; thuật toán cụ thể giống phần trước.

Các giá trị dùng trong bài toán tổng tập con:

- Độ dài quần thể: 20.
- Xác suất chọn: 0.9.
- Xác suất đột biến: 0.1.
- Điều kiện dừng: lời giải tốt nhất hiện tại không đổi sau 100 thế hệ, hoặc đã đạt lời giải tối ưu.

Bài toán TSP. Giả sử có N thành phố, D_{ij} là khoảng cách giữa thành phố i và thành phố j , $D_{ij} = D_{ji}$. Yêu cầu tìm đường đi ngắn nhất qua mọi thành phố và không lặp đường, tức chu trình Hamilton ngắn nhất. Đây là một bài toán NP-đầy đủ điển hình; các phương pháp truyền thống không thật hiệu quả, nên thử dùng thuật toán di truyền.

Ta dùng mã hóa thập phân. Mỗi nhiễm sắc thể gồm dãy số hiệu N thành phố theo một thứ tự nhất định, biểu diễn một hành trình khả dĩ. Độ thích nghi là khoảng cách của hành trình; đường càng ngắn thì độ thích nghi càng cao. Ví dụ, với $N = 10$, nhiễm sắc thể

2 8 4 10 5 1 7 3 6 9

biểu diễn hành trình

$2 \rightarrow 8 \rightarrow 4 \rightarrow 10 \rightarrow 5 \rightarrow 1 \rightarrow 7 \rightarrow 3 \rightarrow 6 \rightarrow 9 \rightarrow 2$.

Tổng độ dài là

$$\sum D_{ij} = D_{28} + D_{84} + D_{4,10} + D_{10,5} + D_{51} + D_{17} + D_{73} + D_{36} + D_{69} + D_{92}.$$

Hàm độ thích nghi có thể định nghĩa bằng biến đổi không âm:

$$f(x) = c_{\max} - \sum D_{ij}.$$

Về chọn lọc, có thể dùng phương pháp chọn lọc xác định đã giới thiệu ở trên. Phép lai ghép ở đây khác với trước: lai ghép đơn giản có thể làm hành trình đi qua cùng một thành phố nhiều lần, vì vậy cần dùng phép lai ghép khớp bộ phận PMX. Cũng có thể bỏ lai ghép và trực tiếp dùng đột biến, vì phép đột biến trong ví dụ này đã hàm chứa thao tác hoán đổi. Với mã thành phố, đột biến là chọn ngẫu nhiên một nhiễm sắc thể, chọn ngẫu nhiên hai gen rồi hoán đổi chúng, tức đảo thứ tự hai thành phố. Vì thuật toán di truyền tối ưu dần dần, có thể dùng thuật toán tham lam để xây dựng quần thể ban đầu.

Các giá trị dùng trong bài toán TSP:

- Độ dài quần thể: 20.

- Xác suất đột biến: 0.1.
- Điều kiện dừng: lời giải tốt nhất hiện tại không đổi sau 100 thế hệ.

Qua hai ví dụ, ta mở rộng định nghĩa đơn giản ban đầu của thuật toán di truyền và giới thiệu ứng dụng của một số kỹ thuật di truyền cao hơn trong các trường hợp cụ thể. Với các bài toán khác nhau cần dùng đối sách khác nhau, bố trí thuật toán hợp lý trong khuôn khổ thuật toán di truyền, cải tiến thuật toán ban đầu hoặc kết hợp với thuật toán sẵn có, để phát huy đầy đủ ưu điểm của thuật toán di truyền.

Kết luận của trang trình chiếu. Nguyên lý của thuật toán di truyền là đơn giản, nhưng dùng thành thạo thuật toán di truyền không hề đơn giản. Lý thuyết phải phù hợp với thực tế; với các bài toán khác nhau, thuật toán di truyền cần thay đổi một chút và không được áp dụng máy móc. Thuật toán di truyền nên kết hợp với các thuật toán tối ưu hiện có để tạo ra thuật toán tốt hơn và thực dụng hơn. Mục đích chính là giúp mọi người có hiểu biết ban đầu về thuật toán di truyền, từ đó thực hành sâu hơn và ứng dụng thuật toán này trong nhiều phương diện.